PYTHON PROJECT

Student Record & Academic Management System

Organizing academic information with Python data structures

Subject	Python
Student Name	Anuj Govind Funde
PRN	2126UDSM1119

A structured and easy-to-follow assignment

1. Objective

The objective of this project is to use Python's built-in data structures to store, organize, update, and retrieve student academic information efficiently.

2. Problem Statement

Managing academic records manually can take time and make it difficult to store, update, search, and retrieve information. Details such as roll number, name, department, subjects, marks, attendance, and contact information need to be arranged in a consistent way.

3. Problem Understanding

The Student Record and Academic Management System organizes student information with suitable Python data structures. It allows records to be stored, updated, searched, and displayed, making academic information easier to manage.

4. Information and System Requirements

Student information to store

- Student name and roll number
- Department and date of birth
- Subjects and marks
- Attendance details
- Contact information

Inputs

- Student details and roll number
- Department and subject
- Marks and attendance
- Contact information

Outputs

- Student records and search results
- Updated records
- Average marks and highest scorer

Relationship between data

Each student record contains several related details. The roll number identifies the student, while the name, department, subjects, marks, attendance, and contact information belong to that student.

Suitable Python data structures

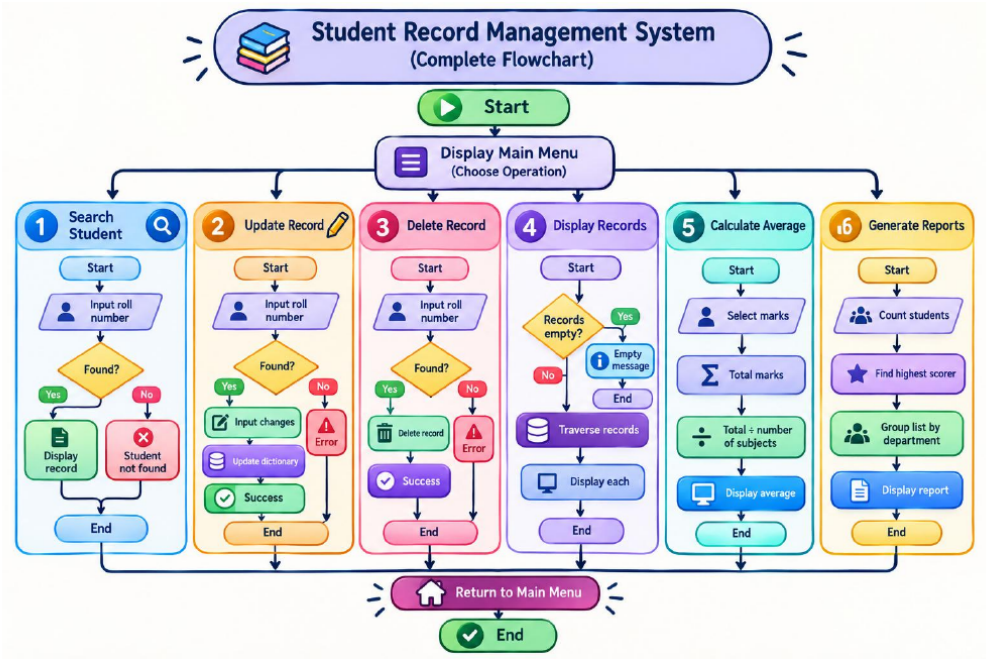
Structure	Use in this project
String	Names, departments, addresses, and email IDs
List	Subjects, marks, attendance, and collections of records
Tuple	Roll number, registration number, and date of birth examples

Structure	Use in this project
Set	Unique departments, subjects, and student clubs
Dictionary	A complete student profile using named keys

5. Flowchart

The original flowchart is retained without changing the diagram.

Flow chart :



Algorithm Design:-

Algorithm 1 :search student by roll no

Purpose: To search student by roll no.

1.Start

2.Enter the student's roll number

3.Search the student records

4.Compare the entered roll number with the stored numbers

5.If the roll number is found the student's complete information

6. Algorithm Design

Algorithm 1: Search for a student by roll number

1. Start.
2. Enter the student’s roll number.
3. Search the student records.
4. Compare the entered roll number with stored roll numbers.
5. If a match is found, display the complete student information.
6. Otherwise, display “Student record not found.”
7. Stop.

Algorithm 2: Add a new student record

1. Start.
2. Enter the student’s roll number, name, registration number, department, and date of birth.
3. Enter subjects and marks for each subject.
4. Enter attendance and contact information.
5. Store all details in a student record.
6. Display “Student record added successfully.”
7. Stop.

Algorithm 3: Update student information

1. Start.
2. Enter the student’s roll number and search for the record.
3. If found, display the current information.
4. Choose the information to update and enter the new value.
5. Replace the old value with the new value.
6. Display “Student record updated.” If the roll number is not found, display “Student record not found.”
7. Stop.

Algorithm 4: Delete a student record

1. Start.
2. Ask for the student’s roll number.
3. Search for the student record.
4. If the record is not found, display “Student not found.”
5. If found, remove the record from the collection.
6. Display “Record deleted successfully.”
7. Stop.

Algorithm 5: Calculate average marks

1. Start.
2. Search for the student record.

3. If the student is not found, display “Student not found.”
4. Retrieve the student’s marks.
5. Add all marks and count the subjects.
6. Calculate average = total marks ÷ number of subjects.
7. Display the average and stop.

Algorithm 6: Find the highest scorer

1. Start.
2. Collect the marks for all students.
3. Assume the first student has the highest marks.
4. Compare the current highest marks with the next student’s marks.
5. If the next marks are higher, update the highest scorer.
6. Continue until every student has been checked.
7. Display the student with the highest marks and stop.

Algorithm 7: Count students

1. Start.
2. Set a counter to zero.
3. Visit each student record.
4. Increase the counter by one for every record.
5. Display the final count and stop.

Algorithm 8: List students by department

1. Start.
2. Ask the user for a department name.
3. Check each student record.
4. Compare the record’s department with the entered department.
5. Display records with a matching department.
6. Continue until all records have been checked, then stop.

7. Pseudocode Development

Searching for a student

```
BEGIN
    INPUT roll_number
    SEARCH student_records for roll_number
    IF roll_number is found THEN
        DISPLAY complete student record
    ELSE
        DISPLAY "Student not found"
    END IF
END
```

Updating student information

```
BEGIN
    INPUT roll_number
    SEARCH student_records for roll_number
    IF roll_number is found THEN
        INPUT new_student_information
        UPDATE selected information
        DISPLAY "Record updated successfully"
    ELSE
        DISPLAY "Student not found"
    END IF
END
```

Deleting a student record

```
BEGIN
    INPUT roll_number
    SEARCH student_records for roll_number
    IF roll_number is found THEN
        DELETE student record
        DISPLAY "Record deleted successfully"
    ELSE
        DISPLAY "Student not found"
    END IF
END
```

Displaying all student records

```
BEGIN
    IF student_records are empty THEN
        DISPLAY "No student records available"
    ELSE
        FOR each student_record
            DISPLAY student information
        END FOR
    END IF
END
```

Calculating average marks

```
BEGIN
    INPUT student_record
    SET total_marks = 0
    SET number_of_subjects = count of subjects
    FOR each subject
        total_marks = total_marks + subject_marks
    END FOR
    average = total_marks / number_of_subjects
    DISPLAY average
END
```

Generating a report

```
BEGIN
    ACCESS all student records
    CALCULATE average marks
    FIND student with highest marks
    LIST students by department
    COUNT total students
    ORGANIZE information into a report
    DISPLAY report
END
```

Adding a new student

```
BEGIN
    INPUT roll_number
    IF roll_number already exists THEN
        DISPLAY "Duplicate record"
    ELSE
        INPUT name, department, subjects, marks
        INPUT attendance, contact_information
        CREATE and STORE student record
        ADD record to student collection
        DISPLAY "Student record added successfully"
    END IF
END
```

8. Python Implementation

The examples below demonstrate how each built-in data structure can represent part of a student record. The sample values are illustrative.

8.1 Strings

```
name = "xyz"
department = "Computer Science"
address = "Aurangabad"
email = "xyz@gmail.com"

name = input("Enter student name: ")
department = input("Enter department: ")

print("Name:", name)
print("Department:", department)
```

8.2 Lists

```
subjects = ["Python", "Maths", "Physics"]
marks = [80, 75, 90]
attendance = [85, 90, 80]

print("Subjects:", subjects)
print("Marks:", marks)
print("Attendance:", attendance)
```

8.3 Tuples

```
roll_number = (101,)
registration_number = ("REG2026CS101",)
date_of_birth = ("15-08-2007",)

print("Roll Number:", roll_number)
print("Registration Number:", registration_number)
print("Date of Birth:", date_of_birth)
```

8.4 Sets

```
departments = {"Computer Science", "Mechanical", "Civil"}
subjects = {"Python", "Maths", "Physics"}
clubs = {"NSS", "Sports", "Coding Club"}

print("Departments:", departments)
print("Subjects:", subjects)
print("Student Clubs:", clubs)
```

8.5 Dictionaries

```
student = {
    "roll number": 101,
    "name": "Rahul",
    "department": "Computer Science",
    "subjects": ["Python", "Maths", "Physics"],
    "marks": [80, 75, 90],
    "attendance": [85, 90, 80],
    "contact": "9876543210"
}

print("Student Record")
print("Roll Number:", student["roll number"])
print("Name:", student["name"])
print("Department:", student["department"])
print("Subjects:", student["subjects"])
print("Marks:", student["marks"])
```

```
    print("Attendance:", student["attendance"])
    print("Contact:", student["contact"])
```

9. Data Structure Integration

Integration means combining strings, lists, tuples, sets, and dictionaries to represent a student record. The dictionary can hold each student's details, while lists store multiple subjects, marks, and attendance values.

```
student = {
    "roll number": (101,),
    "name": "xyz",
    "department": "Computer Science",
    "subjects": ["Python", "Maths", "Physics"],
    "marks": [80, 75, 90],
    "attendance": [85, 90, 80],
    "contact": "9876543210"
}
```

10. Testing Report

Test case	Input / condition	Expected result
Add student	Valid student details	Student added successfully
Search student	Existing roll number	Student record displayed
Search student	Non-existing roll number	“Student not found”
Delete student	Existing roll number	Record deleted
Calculate average	Valid marks	Average displayed
Duplicate record	Roll number already exists	Duplicate record rejected
Empty records	No student records	“No records available”

11. Observations

- The system stores student information in an organized way.
- Different structures serve different purposes: strings for text, lists for collections, tuples for fixed example values, sets for unique values, and dictionaries for complete profiles.
- The system design covers adding, searching, updating, deleting, and displaying student records.
- It can calculate average marks and identify the highest scorer.
- Department-wise records and the total student count can be included in reports.
- Testing considers duplicate entries, missing records, empty collections, and other possible errors.
- Combining Python data structures provides a clear model for managing academic information.
- The project demonstrates how Python collections can be applied to a practical record-management task.

Conclusion

This assignment shows how Python’s built-in data structures can be combined to organize student details and support common academic record operations. The algorithms, pseudocode, implementation examples, and test cases describe the main parts of the proposed system.